Software Requirements Specification

HYBRID_LIB_ADA

Document Control

Field	Value
Author(s)	Michael Gardner
Version	2.1.0
Status	Released
Status Date	2025-12-14
SPDX-License-Identifier	BSD-3-Clause
License File	See the LICENSE file in the project root
Copyright	© 2025 Michael Gardner, A Bit of Help, Inc.

Project Profile

Field	Value
Variant	library
Library Role	enterprise
Assurance	spark-targeted
Execution	sequential
Parallelism	none
Platform	desktop, server, embedded
Execution Environment	linux, macos, windows, ada-runtime
Processor Architecture	amd64, arm64, arm32
Deployment	native

Table of Contents

1. Introduction 1
   1.1. Purpose 1
   1.2. Scope 1
   1.3. Definitions and Acronyms 1
   1.4. References 1
2. Overall Description 1
   2.1. Product Perspective 1
   2.2. Product Features 2
   2.3. User Classes 2
   2.4. Operating Environment 2
   2.5. Constraints 2
3. Interface Requirements 2
   3.1. User Interfaces 2
   3.2. Software Interfaces 3
      3.2.1. Alire Integration 3
      3.2.2. Ada API 3
   3.3. Hardware Interfaces 3
4. Functional Requirements 3
   4.1. Domain Layer (FR-01) 3
   4.2. Application Layer (FR-02) 3
   4.3. Infrastructure Layer (FR-03) 4
   4.4. API Layer (FR-04) 4
   4.5. Error Handling (FR-05) 4
5. Quality and Cross-Cutting Requirements 5
   5.1. Performance (NFR-01) 5
   5.2. Reliability (NFR-02) 5
   5.3. Portability (NFR-03) 5
   5.4. Maintainability (NFR-04) 5
   5.5. Usability (NFR-05) 5
   5.6. Platform Abstraction (NFR-06) 5
   5.7. SPARK Formal Verification (NFR-07) 6
   5.8. Testability (NFR-08) 6
6. Design and Implementation Constraints 6
   6.1. System Requirements 6
      6.1.1. Hardware Requirements 6
      6.1.2. Software Requirements 7
   6.2. Architectural Constraints 7
7. Verification and Traceability 7
   7.1. Verification Methods 7
   7.2. Traceability Matrix 7
8. Appendices 8
   8.1. Glossary 8
   8.2. Change History 8

1. Introduction

1.1. Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for **Hybrid_Lib_Ada**, a canonical Ada 2022 library demonstrating hybrid DDD/Clean/Hexagonal architecture with functional error handling.

1.2. Scope

Hybrid_Lib_Ada provides:

- A reusable library for greeting operations.
- Demonstration of DDD/Clean/Hexagonal architecture in Ada.
- Functional error handling via a Result monad.
- SPARK-compatible design for formal verification.
- Embedded-safe patterns that avoid heap allocation.

1.3. Definitions and Acronyms

Term	Definition
DDD	Domain-Driven Design — strategic and tactical patterns for complex software.
Hexagonal Architecture	Ports & Adapters pattern isolating business logic from infrastructure.
Result Monad	Functional pattern for error handling without exceptions.
SPARK	Ada subset for formal verification.
Value Object	Immutable domain object defined by its attributes.

1.4. References

- Ada 2022 Reference Manual (ISO/IEC 8652:2023).
- SPARK 2014 Reference Manual.
- **Domain-Driven Design** (Eric Evans, 2003).
- **Clean Architecture** (Robert C. Martin, 2017).

2. Overall Description

2.1. Product Perspective

Hybrid_Lib_Ada is a standalone library designed to be imported by Ada applications implementing hexagonal (ports and adapters) architecture with clean separation between domain logic, application use cases, and infrastructure adapters.

This library profile uses the same hybrid Domain/Application/Infrastructure core as **Hybrid_App_Ada**, but expresses the outer boundary through `API`, `API.Desktop`, and `API.Operations` rather than `Presentation` and `Bootstrap`.

Client Application
`with Hybrid_Lib_Ada.API;`
`Result := API.Greet (API.Create_Greet_Command("X"));`

↓

Hybrid_Lib_Ada
API Layer ⟶ Application Layer ⟶ Domain Layer
Infrastructure Layer (adapters)

Supporting architecture guidance is maintained in `docs/guides/`. Supporting UML diagrams are maintained in `docs/diagrams/`.

Layer	Purpose
Domain	Pure business logic, value objects, error types.
Application	Use cases, commands, ports (interfaces).
Infrastructure	Adapters for I/O operations.
API	Public facade with stable interface.

2.2. Product Features

1. **Greeting Operations**: Generate personalized greetings via clean architecture.
2. **Domain Modeling**: Provide a Person value object with validation.
3. **Error Handling**: Use railway-oriented programming with Result monads.
4. **Platform Abstraction**: Provide a generic I/O plugin pattern for portability.
5. **Formal Verification**: Maintain a SPARK-compatible domain layer.

2.3. User Classes

User Class	Description
Library Consumers	Ada developers integrating library functionality.
Architecture Students	Developers learning hexagonal architecture in Ada.
Embedded Developers	Developers requiring heap-free, SPARK-compatible patterns.

2.4. Operating Environment

Requirement	Specification
Platforms	POSIX (Linux, macOS, BSD), Windows 11, Embedded.
Ada Compiler	GNAT FSF 13+ or GNAT Pro.
Ada Version	Ada 2022.
Dependencies	`functional ^3.0.0`

2.5. Constraints

Constraint	Rationale
Ada 2022	Required for modern language features.
GNAT 13+	Required compiler version.
No Heap Allocation	Embedded system compatibility.
SPARK Subset	Formal verification capability.

3. Interface Requirements

3.1. User Interfaces

None. This is a library, not an application.

3.2. Software Interfaces

3.2.1. Alire Integration

```
[[depends-on]]
hybrid_lib_ada = "*"
```

3.2.2. Ada API

```
with Hybrid_Lib_Ada.API;
use Hybrid_Lib_Ada.API;

Cmd    : constant Greet_Command := Create_Greet_Command ("Name");
Result : constant Unit_Result.Result := Greet (Cmd);
```

3.3. Hardware Interfaces

None. The library is hardware-agnostic.

4. Functional Requirements

4.1. Domain Layer (FR-01)

Priority: High **Description:** Provide core domain entities and value objects.

ID	Requirement
FR-01.1	Person value object with bounded name (1–100 characters).
FR-01.2	Person is immutable after creation.
FR-01.3	Empty names rejected with `Validation_Error`.
FR-01.4	Structured error types with `Kind` enumeration and `Message`.
FR-01.5	Result monad for all fallible operations.
FR-01.6	Error kinds: `Validation_Error`, `Parse_Error`, `Not_Found_Error`, `IO_Error`, `Internal_Error`.

Acceptance Criteria (FR-01):

- Person value object validates name on creation.
- Result is either `Ok(value)` or `Error(error_info)`.
- No exceptions are raised for expected errors.
- Value extraction remains type-safe.

4.2. Application Layer (FR-02)

Priority: High **Description:** Provide use cases and port definitions.

ID	Requirement
FR-02.1	Provide `Greet_Command` data transfer object for greeting requests.
FR-02.2	Provide Greet use case orchestrating the greeting workflow.
FR-02.3	Define `Writer` outbound port for output operations.
FR-02.4	Use static polymorphism via generic function signatures.
FR-02.5	Return `Result[Unit]` for all operations.

Acceptance Criteria (FR-02):

- `Greet_Command` encapsulates the name and remains immutable after creation.

- The Greet use case accepts a command, creates a `Person`, generates the greeting, and writes through the port.
- The `Writer` port provides a generic signature equivalent to `Write(Message) -> Result[Unit]`.
- The port is Application-owned and Infrastructure-implemented.

4.3. Infrastructure Layer (FR-03)

Priority: High **Description:** Provide platform-specific adapters implementing ports.

ID	Requirement
FR-03.1	Provide `Console_Writer` adapter for standard output.
FR-03.2	Implement the `Writer` port contract.
FR-03.3	Return `Ok` on success and `IO_Error` on failure.

Acceptance Criteria (FR-03):

- `Console_Writer` writes to standard output.
- All I/O errors are captured and returned as Result values.

4.4. API Layer (FR-04)

Priority: High **Description:** Provide stable public interface.

ID	Requirement
FR-04.1	Expose a single `Hybrid_Lib_Ada.API` package for imports.
FR-04.2	Re-export domain types such as `Person`, `Error`, and `Unit`.
FR-04.3	Re-export application types such as `Greet_Command` and `Unit_Result`.
FR-04.4	Provide `API.Operations` with `SPARK_Mode(On)` for verifiable logic.
FR-04.5	Provide `API.Desktop` composition root wiring `Console_Writer`.

Acceptance Criteria (FR-04):

- `API.Operations` is generic and parameterized by the `Writer` port.
- `Operations` introduces no Infrastructure dependencies.
- `API.Desktop` wires `Console_Writer` and uses `SPARK_Mode(Off)` for I/O wiring.

4.5. Error Handling (FR-05)

Priority: High **Description:** Railway-oriented error handling without exceptions.

ID	Requirement
FR-05.1	Use Result monad for all fallible operations.
FR-05.2	Provide descriptive error messages.
FR-05.3	Provide error codes for all failure modes.
FR-05.4	Do not use exceptions in library code for expected failures.

Error Handling Policy Note: Expected failures shall be represented as `Result` errors rather than propagated exceptions. Unexpected runtime failures (for example `Program_Error` or `Storage_Error`) remain outside this policy and shall be contained at architectural boundaries using `Functional.Try` or equivalent boundary handling.

5. Quality and Cross-Cutting Requirements

5.1. Performance (NFR-01)

ID	Requirement
NFR-01.1	Operation latency < 1 ms (excluding I/O).
NFR-01.2	Zero heap allocation.
NFR-01.3	Stack usage < 4 KB per call.

5.2. Reliability (NFR-02)

ID	Requirement
NFR-02.1	Return all errors via Result monad with no exceptions for expected failures.
NFR-02.2	Validate all inputs at the domain boundary.
NFR-02.3	Permit no uncaught exceptions for expected error paths.

5.3. Portability (NFR-03)

ID	Requirement
NFR-03.1	Support POSIX platforms (Linux, macOS, BSD).
NFR-03.2	Support Windows platforms.
NFR-03.3	Support embedded platforms via custom adapters.
NFR-03.4	Keep platform-specific code out of the domain layer.
NFR-03.5	Expose no infrastructure types in application-layer ports.
NFR-03.6	Select platform adapters via GPR configuration.

5.4. Maintainability (NFR-04)

ID	Requirement
NFR-04.1	Use hexagonal architecture with clear boundaries.
NFR-04.2	Provide comprehensive documentation and docstrings.
NFR-04.3	Maintain > 90% test coverage.
NFR-04.4	Maintain zero compiler warnings.

5.5. Usability (NFR-05)

ID	Requirement
NFR-05.1	Provide a clear, intuitive API.
NFR-05.2	Provide working examples for all use cases.
NFR-05.3	Provide comprehensive error messages.

5.6. Platform Abstraction (NFR-06)

ID	Requirement
NFR-06.1	Application layer defines abstract outbound ports using pure function signatures.
NFR-06.2	Infrastructure layer provides platform-specific adapters implementing ports.
NFR-06.3	Composition roots (`API.Desktop`, `API.Windows`, `API.Embedded`) wire adapters to ports.

ID	Requirement
NFR-06.4	Use domain types in port signatures, not infrastructure types.
NFR-06.5	Add new platforms without modifying the application layer.
NFR-06.6	Keep all platform adapters testable via mock implementations.

5.7. SPARK Formal Verification (NFR-07)

ID	Requirement
NFR-07.1	Domain layer shall pass SPARK legality checking.
NFR-07.2	All domain packages shall use `SPARK_Mode => On`.
NFR-07.3	Permit no provable runtime errors in the domain layer (overflow, range, division).
NFR-07.4	Require all domain variables to be initialized before use.
NFR-07.5	Require preconditions and postconditions on domain operations to be provably correct.
NFR-07.6	Run SPARK legality verification via `make spark-check`.
NFR-07.7	Run SPARK proof verification via `make spark-prove`.
NFR-07.8	Infrastructure and API layers may use `SPARK_Mode => Off` for I/O operations.

Layer	SPARK_Mode	Rationale
Domain	On	Pure business logic, provable.
Application	On	Operations, inbound ports, outbound ports.
Infrastructure	Off	I/O operations.
API	Off	Facade over infrastructure.

5.8. Testability (NFR-08)

ID	Requirement
NFR-08.1	Provide unit tests for all domain packages.
NFR-08.2	Provide integration tests for cross-layer interactions.
NFR-08.3	Provide mock adapters for testing without I/O.
NFR-08.4	Provide test runners with pass/fail reporting.

6. Design and Implementation Constraints

6.1. System Requirements

6.1.1. Hardware Requirements

Category	Requirement
CPU	Any modern processor.
RAM	64 MB minimum.
Disk	10 MB minimum.

6.1.2. Software Requirements

Category	Requirement
Operating System	Linux, macOS, BSD, Windows 11.
Compiler	GNAT FSF 13+ or GNAT Pro.
Build System	Alire 2.0+.

6.2. Architectural Constraints

- Maintain domain isolation from infrastructure.
- Keep application-owned ports free from infrastructure types.
- Preserve composition-root responsibility for adapter wiring.
- Document any project-specific exceptions explicitly.

Architecture conformance shall be enforced through a combination of project/build configuration constraints and automated validation using `scripts/arch_guard.py`. The architecture guard shall be run during normal developer build workflow to detect direct dependency violations early and in CI/CD to prevent non-conforming changes from being accepted.

7. Verification and Traceability

7.1. Verification Methods

Method	Description
Code Review	All code reviewed before merge.
Static Analysis	Zero compiler warnings.
Dynamic Testing	All tests must pass.
Coverage Analysis	> 90% line coverage.
Architecture Validation	`arch_guard.py` validates direct dependency rules during developer builds and CI/CD; project/build interface restrictions provide additional enforcement where configured.

7.2. Traceability Matrix

Requirement	Design	Test
FR-01.1	Domain.Value_Object.Person	test_domain_person.adb
FR-01.4	Domain.Error	test_domain_error_result.adb
FR-01.5	Domain.Error.Result	test_domain_error_result.adb
FR-02.1	Application.Command.Greet	test_application_command_greet.adb
FR-02.2	Application.Usecase.Greet	test_application_usecase_greet.adb
FR-02.3	Application.Port.Outbound.Writer	test_application_usecase_greet.adb
FR-03.1	Infrastructure.Adapter.Console_Writer	test_api_greet.adb
FR-04.1	Hybrid_Lib_Ada.API	test_api_greet.adb
FR-04.4	Hybrid_Lib_Ada.API.Operations	test_api_operations.adb
FR-04.5	Hybrid_Lib_Ada.API.Desktop	test_api_greet.adb

8. Appendices

8.1. Glossary

See Section 1.3 Definitions and Acronyms.

8.2. Change History

Version	Date	Author	Changes
2.1.0	2025-12-14	Michael Gardner	Remove project statistics section; metrics belong in CHANGELOG. Add Section 6.2 Architectural Constraints and explicit architecture-enforcement language.
2.0.0	2025-12-09	Michael Gardner	Complete regeneration for v2.0.0; corrected Error_Kind values; added NFR-08 Testability.
1.0.0	2025-12-08	Michael Gardner	Initial release.